```
puts x
```

*produces:*

```
2
```

Note that the return value of the rightward assignment itself is nil, not the value of the assignment.

## Parallel Assignment

You may have had to write code to swap the values in two variables, and done so by creating a temporary variable:

```
# C, or Java, or ...
int a = 1;
int b = 2;
int temp;

temp = a;
a = b;
b = temp;
```

You can do this swap much more cleanly in Ruby with parallel assignment:

```
a, b = 1, 2    # a=1, b=2
a, b = b, a    # b=2, a=1
```

Ruby lets you have a comma-separated list of rvalues (the things on the right of the assignment). Once Ruby sees more than one rvalue in an assignment, the rules of parallel assignment come into play. What follows is a description at the logical level: what happens inside the interpreter is somewhat hairier.

When Ruby interprets a parallel assignment, the values on the left are evaluated before the values on the right.

Then all the rvalues are evaluated, left to right, and collected into an array (unless they are already an array). This array will be the eventual value returned by the overall assignment.

Next, the left side result is inspected. If it contains a single element, the array is assigned to that element.

```
a = 1, 2, 3, 4      # a=[1, 2, 3, 4]
b = [1, 2, 3, 4]    # b=[1, 2, 3, 4]
```

If the left side contains a comma, Ruby matches values on the right side against successive elements on the left side. Excess elements are discarded.

```
a, b = 1, 2, 3, 4    # a=1, b=2
c, = 1, 2, 3, 4      # c=1
```

## Splats and Assignment

If Ruby sees any splats on the right side of an assignment (that is, rvalues preceded by an asterisk), each will be expanded inline into its constituent values during the evaluation of the rvalues and before the assignment to lvalues starts: